

尚硅谷大数据技术之 Hadoop (优化&新特性)

(作者: 尚硅谷研究院)

版本: V5.0

第 1 章 HDFS—Datanode 管理多块数据盘

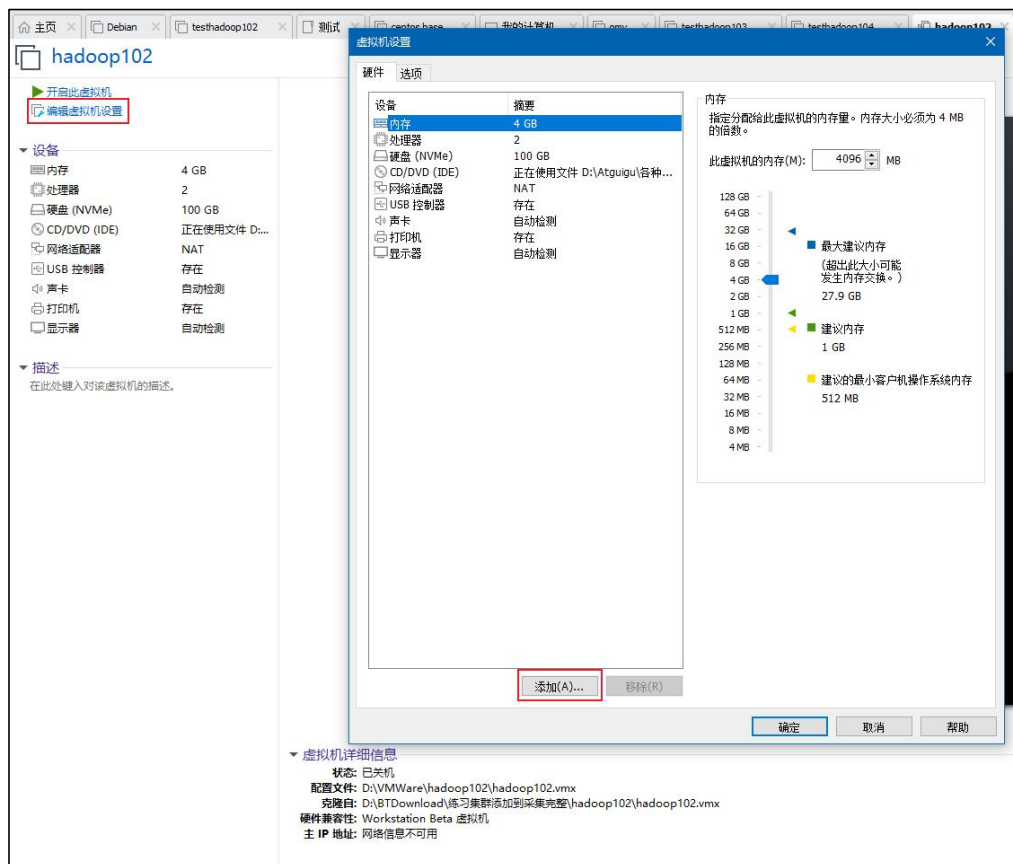
实际生产中,一台服务器往往不止一块硬盘。而 HDFS 在实际应用中,往往会采取数据分离存储:也就是将框架本体和数据分开存放,数据要专门存储到单独的数据盘中。当集群出现问题,可以将数据盘单独取出来恢复数据。

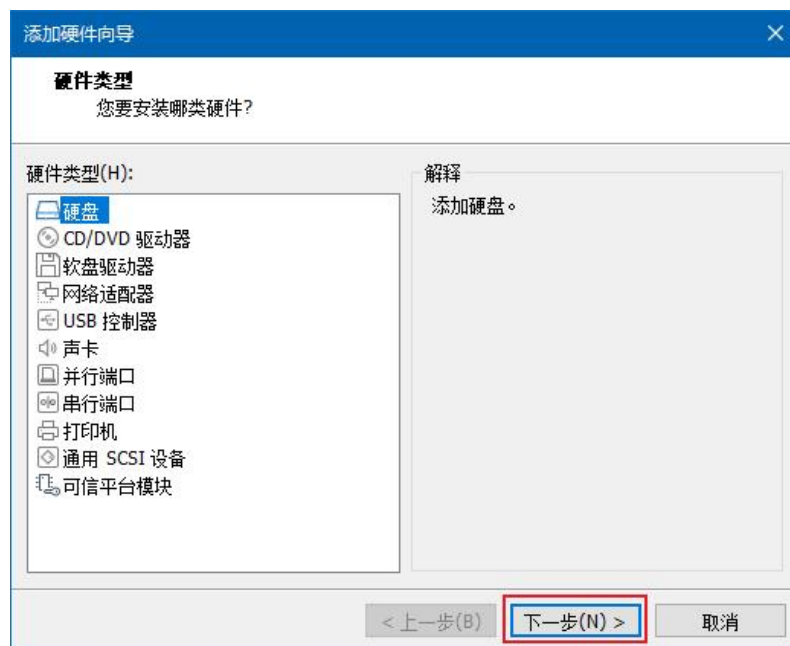
为了模拟这种情况,接下来我们会给 hadoop102 添加多块硬盘,并将它们全部配置为 HDFS 存储。

1.1 Hadoop102 添加磁盘

首先关闭 hadoop102 虚拟机,在进行下一步之前,照一个快照,方便之后恢复。

按照如下图示添加硬盘:





添加硬件向导

选择磁盘

您要使用哪个磁盘?

磁盘

☒ 创建新虚拟磁盘(V)
虚拟磁盘由主机文件系统上的一个或多个文件组成，客户机操作系统会将其视为单个硬盘。虚拟磁盘可在一台主机上或多台主机之间轻松复制或移动。

☐ 使用现有虚拟磁盘(E)
选择此选项可重新使用以前配置的磁盘。

☐ 使用物理磁盘 (适用于高级用户)(P)
选择此选项可为虚拟机提供直接访问本地硬盘的权限。需要具有管理员特权。

< 上一步(B)

下一步(N) >

取消

添加硬件向导

指定磁盘容量

磁盘大小为多少?

最大磁盘大小 (GB)(S): 20.0

针对 CentOS 7 64 位的建议大小: 20 GB

☐ 立即分配所有磁盘空间(A)。
分配所有容量可以提高性能，但要求所有物理磁盘空间立即可用。如果不立即分配所有空间，虚拟磁盘的空间最初很小，会随着您向其中添加数据而不断变大。

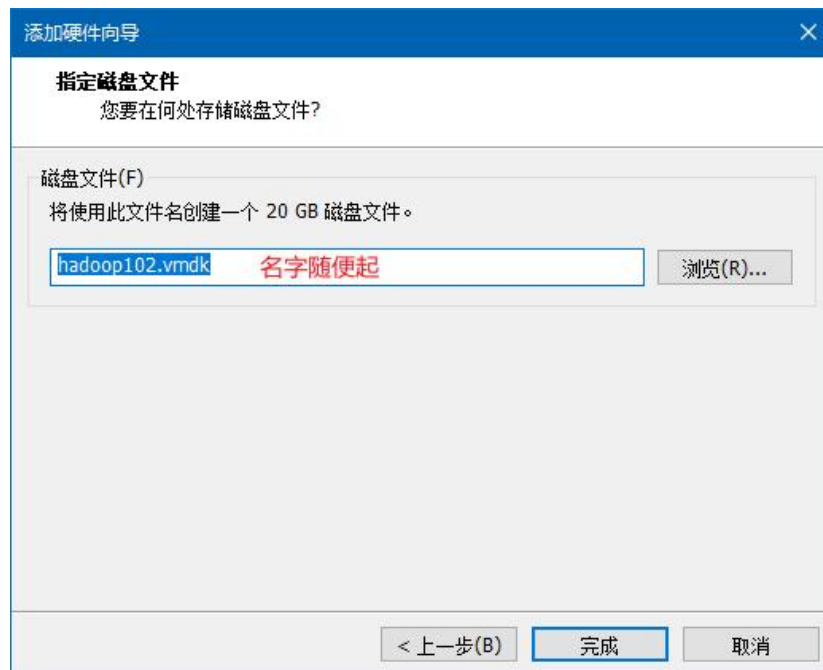
☒ 将虚拟磁盘存储为单个文件(O)

☐ 将虚拟磁盘拆分成多个文件(M)
拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。

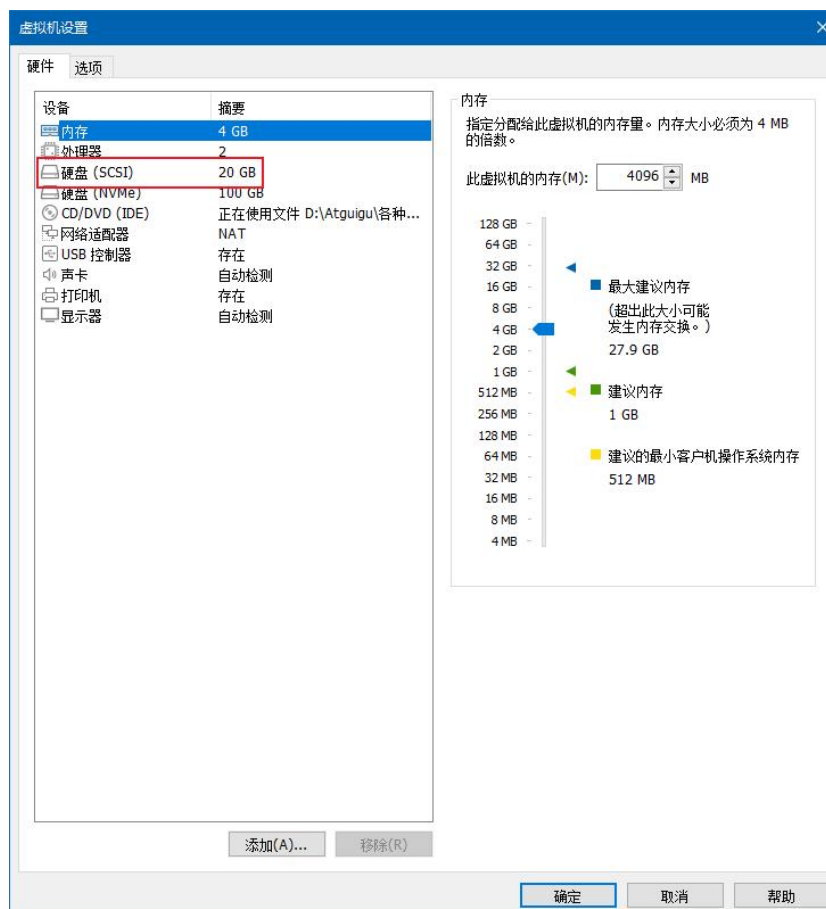
< 上一步(B)

下一步(N) >

取消



完成后能看到虚拟机中多了一块硬盘：



之后打开虚拟机，查看刚刚添加的磁盘对应的设备。

```
[atguigu@hadoop102 ~]$ lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
```

```
sda                8:0    0   20G   0 disk
sr0                11:0    1   973M   0 rom
nvme0n1           259:0    0   100G   0 disk
├─nvme0n1p1        259:1    0    1G    0 part  /boot
└─nvme0n1p2        259:2    0   99G    0 part
   ├─centos-root    253:0    0  95.1G   0 lvm   /
   └─centos-swap    253:1    0   3.9G   0 lvm   [SWAP]
```

可以看到我们刚刚添加的磁盘叫 **sda**（每个人的虚拟机可能不一样，总之找到新出现的 20G 磁盘，就是我们刚刚添加的磁盘）。此时这个磁盘没有格式化，也没有挂载，是不能访问的。

接下来执行以下指令对磁盘进行格式化：

```
[atguigu@hadoop102 ~]$ fdisk /dev/sdb
欢迎使用 fdisk (util-linux 2.23.2)。
```

更改将停留在内存中，直到您决定将更改写入磁盘。
使用写入命令前请三思。

Device does not contain a recognized partition table
使用磁盘标识符 0x457af815 创建新的 DOS 磁盘标签。

命令 (输入 m 获取帮助)：

按 **n** 新建分区，并不断回车接受默认配置。完成后按 **w** 保存磁盘分区：

命令 (输入 m 获取帮助)： **n**

Partition type:

```
p  primary (0 primary, 0 extended, 4 free)
e  extended
```

Select (default p):

Using default response p

分区号 (1-4, 默认 1):

起始 扇区 (2048-41943039, 默认为 2048):

将使用默认值 2048

Last 扇区, +扇区 or +size{K,M,G} (2048-41943039, 默认为 41943039):

将使用默认值 41943039

分区 1 已设置为 Linux 类型，大小设为 20 GiB

命令 (输入 m 获取帮助)： **w**

The partition table has been altered!

Calling ioctl() to re-read partition table.

正在同步磁盘。

完成后重新查看分区情况，可以看到我们刚刚添加的分区。

```
[atguigu@hadoop102 ~]$ lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda                  8:0    0   20G   0 disk
├─sda1                8:1    0   20G   0 part
sr0                  11:0    1   973M   0 rom
nvme0n1             259:0    0   100G   0 disk
├─nvme0n1p1          259:1    0    1G    0 part  /boot
```

```
└─nvme0n1p2    259:2    0   99G  0 part
└─centos-root 253:0    0  95.1G  0 lvm  /
└─centos-swap 253:1    0   3.9G  0 lvm  [SWAP]
```

执行 `mkfs.ext4` 指令格式化刚刚建立的分区。

```
[atguigu@hadoop102 ~]$ sudo mkfs.ext4 /dev/sdb1
mke2fs 1.42.9 (28-Dec-2013)
文件系统标签=
OS type: Linux
块大小=4096 (log=2)
分块大小=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
1310720 inodes, 5242624 blocks
262131 blocks (5.00%) reserved for the super user
第一个数据块=0
Maximum filesystem blocks=2153775104
160 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632,
    2654208,
    4096000

Allocating group tables: 完成
正在写入 inode 表: 完成
Creating journal (32768 blocks): 完成
Writing superblocks and filesystem accounting information: 完成
```

格式化完成后，最后新建一个目录挂载这个磁盘即可。

```
[atguigu@hadoop102 ~]$ sudo mkdir -p /data/disk1
[atguigu@hadoop102 ~]$ sudo mount /dev/sdb1 /data/disk1
[atguigu@hadoop102 ~]$ lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda                  8:0    0   20G  0 disk
└─sda1               8:1    0   20G  0 part /data/disk1
sr0                  11:0    1   973M  0 rom
nvme0n1             259:0    0  100G  0 disk
└─nvme0n1p1         259:1    0    1G  0 part /boot
└─nvme0n1p2         259:2    0   99G  0 part
└─centos-root       253:0    0  95.1G  0 lvm  /
└─centos-swap       253:1    0   3.9G  0 lvm  [SWAP]
```

可以看到我们的磁盘挂载在了 `/data/disk1` 位置。此时访问这个目录，实际访问的就是我们新建的磁盘。

由于这个磁盘目前 `atguigu` 用户没有权限访问，最后改一次权限即可：

```
[atguigu@hadoop102 ~]$ sudo chown -R atguigu:atguigu /data/disk1
[atguigu@hadoop102 ~]$ ll /data
总用量 4
drwxr-xr-x. 3 atguigu atguigu 4096 12月  1 04:29 disk1
```

1.2 配置 DataNode 管理多磁盘

目前 hadoop102 的 datanode 并不能识别到这块磁盘。具体信息我们可以查看 <http://hadoop102:9864> 查看：

HadoopOverviewUtilities

DataNode on hadoop102:9866

Cluster ID:

CID-b8ca4776-a350-4143-9f5c-9f36536e81b7

Version:

3.1.3, rba631c436b806728f8ec2f54ab1e289526c90579

Block Pools

Namenode Address	Block Pool ID	Actor State	Last Heartbeat	Last Block Report	Last Block Report Size (Max Size)
hadoop102:9820	BP-701651822-192.168.10.102-1646946569438	RUNNING	1s	2 minutes	0 B (64 MB)

Volume Information

Directory	StorageType	Capacity Used	Capacity Left	Capacity Reserved	Reserved Space for Replicas	Blocks
/opt/module/hadoop/data/dfs/data	DISK	4 KB	92.49 GB	0 B	0 B	0

Hadoop, 2019.

为了方便一会实验多磁盘负载均衡，我们此时可以给 HDFS 集群上传大量的文件。

```
[atguigu@hadoop102 ~]$ sudo tar -zcf usr.tar.gz /usr
[atguigu@hadoop102 ~]$ hadoop fs -put usr.tar.gz /1.tar.gz
[atguigu@hadoop102 ~]$ hadoop fs -put usr.tar.gz /2.tar.gz
[atguigu@hadoop102 ~]$ hadoop fs -put usr.tar.gz /3.tar.gz
[atguigu@hadoop102 ~]$ hadoop fs -put usr.tar.gz /4.tar.gz
[atguigu@hadoop102 ~]$ hadoop fs -put usr.tar.gz /5.tar.gz
```

此时再次查看 hadoop102 datanode 状态：

Hadoop

Overview

Utilities

DataNode on hadoop102:9866

Cluster ID:

CID-b8ca4776-a350-4143-9f5c-9f36536e81b7

Version:

3.1.3, rba631c436b806728f8ec2f54ab1e289526c90579

Block Pools

Namenode Address

Block Pool ID

Actor State

Last Heartbeat

Last Block Report

Last Block Report Size (Max Size)

hadoop102:9820

BP-701651822-192.168.10.102-1646946569438

RUNNING

2s

12 minutes

0 B (64 MB)

Volume Information

Directory

StorageType

Capacity Used

Capacity Left

Capacity Reserved

Reserved Space for Replicas

Blocks

/opt/module/hadoop/data/dfs/data

DISK

2.48 GB

89.66 GB

0 B

0 B

20

Hadoop, 2019.

接下来我们可以配置 hadoop102 的 datanode 识别第二块磁盘，这里要用到 HDFS 的多目录配置。

1) DataNode 可以配置成多个目录，每个目录存储的数据不一样（数据不是副本）



2) 具体配置如下

在 hdfs-site.xml 文件中添加如下内容。

```
<property>
  <name>dfs.datanode.data.dir</name>
  <value>file://${hadoop.tmp.dir}/dfs/data,file:///data/disk1</value>
</property>
```

3) 完成后重启 hadoop102 的 datanode，并查看结果：

```
[atguigu@hadoop102 ~]$ hdfs --daemon start datanode
```

Hadoop

Overview

Utilities

DataNode on hadoop102:9866

Cluster ID:	CID-b8ca4776-a350-4143-9f5c-9f36536e81b7
Version:	3.1.3, rba631c436b806728f8ec2f54ab1e289526c90579

Block Pools

Namenode Address	Block Pool ID	Actor State	Last Heartbeat	Last Block Report	Last Block Report Size (Max Size)
hadoop102:9820	BP-701651822-192.168.10.102-1646946569438	RUNNING	1s	a few seconds	240 B (64 MB)

Volume Information

Directory	StorageType	Capacity Used	Capacity Left	Capacity Reserved	Reserved Space for Replicas	Blocks
/opt/module/hadoop/data/dfs/data	DISK	2.48 GB	89.66 GB	0 B	0 B	20
/data/disk1	DISK	24 KB	18.5 GB	0 B	0 B	0

Hadoop, 2019.

此时 datanode 已成功识别两块磁盘。接下来上传的数据块，会均匀分布在两块磁盘里，以充分利用磁盘性能。

1.3 集群数据均衡之磁盘间数据均衡

实际生产中，如果因为磁盘空间不足而添加新磁盘，会产生一个问题：新老磁盘占用的百分比不同，也就是多磁盘间数据不均衡。这样会导致所有新写入集群的数据（热数据）绝大部分流入新磁盘，此时我们只能发挥一块磁盘的性能。此时我们需要做多磁盘的数据均衡，让多块磁盘之间的空间占比基本一致。（Hadoop3.x 新特性）。



生成均衡计划

```
[atguigu@hadoop102 ~]$ hdfs diskbalancer -plan hadoop102
--thresholdPercentage 1 -v
2022-12-01 05:10:43,905 INFO planner.GreedyPlanner: Starting plan for
Node : hadoop102:9867
2022-12-01 05:10:43,907 INFO planner.GreedyPlanner: Disk Volume set
b3a62d40-8a7d-42cc-94a7-22efab7168d0 Type : DISK plan completed.
2022-12-01 05:10:43,907 INFO planner.GreedyPlanner: Compute Plan for
Node : hadoop102:9867 took 14 ms
```

```
2022-12-01 05:10:43,907 INFO command.Command: Writing plan to:
2022-12-01 05:10:43,907 INFO command.Command:
/system/diskbalancer/2022-十二月-01-05-10-43/hadoop102.plan.json

Plan :

=====
Source Disk                               Dest.Disk                               Size
Type
/opt/module/hadoop/data/dfs/data/         /data/disk1/
410.6 M      DISK
=====
Writing plan to:
/system/diskbalancer/2022-十二月-01-05-10-43/hadoop102.plan.json
```

执行均衡计划

```
[atguigu@hadoop102 ~]$ hdfs diskbalancer -execute
/system/diskbalancer/2022-十二月-01-05-10-43/hadoop102.plan.json
```

查看当前均衡任务的执行情况

```
[atguigu@hadoop102 ~]$ hdfs diskbalancer -query hadoop102
2022-12-01 05:13:30,004 INFO command.Command: Executing "query plan"
command.
Plan      File:      /system/diskbalancer/2022- 十 二 月
-01-05-10-43/hadoop102.plan.json
Plan ID: 777574d601f0c91a46c699e275fa178681d66780
Result: PLAN_UNDER_PROGRESS
```

等到任务完成查看均衡结果

Hadoop
Overview
Utilities

DataNode on hadoop102:9866

Cluster ID:	CID-b8ca4776-a350-4143-9f5c-9f36536e81b7
Version:	3.1.3, rba631c436b806728f8ec2f54ab1e289526c90579

Block Pools

Namenode Address	Block Pool ID	Actor State	Last Heartbeat	Last Block Report	Last Block Report Size (Max Size)
hadoop102:9820	BP-701651822-192.168.10.102-1646946569438	RUNNING	2s	20 minutes	240 B (64 MB)

Volume Information

Directory	StorageType	Capacity Used	Capacity Left	Capacity Reserved	Reserved Space for Replicas	Blocks
/opt/module/hadoop/data/dfs/data	DISK	1.98 GB	90.04 GB	0 B	0 B	18
/data/disk1	DISK	387.06 MB	18.12 GB	0 B	0 B	4

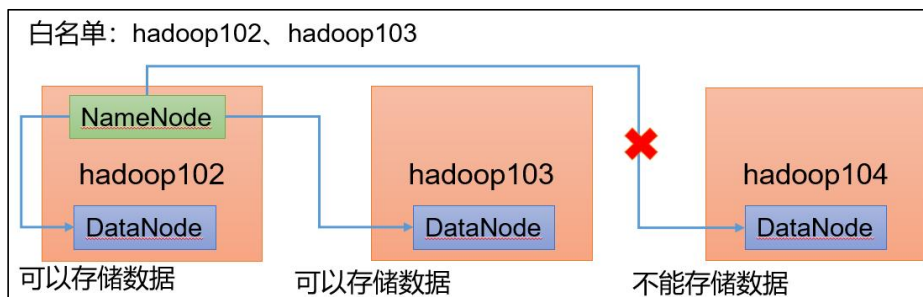
Hadoop, 2019.

第 2 章 HDFS—集群扩容及缩容

2.1 添加白名单

白名单：表示在白名单的主机 IP 地址可以，用来存储数据。

企业中：配置白名单，可以尽量防止黑客恶意访问攻击。



配置白名单步骤如下：

- 1) 在 NameNode 节点的/opt/module/hadoop-3.3.4/etc/hadoop 目录下分别创建 whitelist 和 blacklist 文件

创建白名单

```
[atguigu@hadoop102 hadoop]$ vim whitelist
```

在 whitelist 中添加如下主机名称，假如集群正常工作的节点为 102 103

```
hadoop102
hadoop103
```

创建黑名单

```
[atguigu@hadoop102 hadoop]$ touch blacklist
```

保持空的就可以。

- 2) 在 hdfs-site.xml 配置文件中增加 dfs.hosts 配置参数

```
<!-- 白名单 -->
<property>
  <name>dfs.hosts</name>
  <value>/opt/module/hadoop-3.3.4/etc/hadoop/whitelist</value>
</property>

<!-- 黑名单 -->
<property>
  <name>dfs.hosts.exclude</name>
  <value>/opt/module/hadoop-3.3.4/etc/hadoop/blacklist</value>
</property>
```

- 3) 第一次添加白名单必须重启集群，不是第一次，只需要刷新 NameNode 节点即可

```
[atguigu@hadoop102 hadoop-3.3.4]$ myhadoop.sh stop
[atguigu@hadoop102 hadoop-3.3.4]$ myhadoop.sh start
```

- 4) 在 web 浏览器上查看 DN，<http://hadoop102:9870/dfshealth.html#tab-datanode>

Show 25 entries

Search:

Node	Http Address	Last contact	Last Block Report	Capacity	Blocks	Block pool used	Version
✓hadoop102:9866 (192.168.10.102:9866)	http://hadoop102:9864	0s	2m	89.95 GB	3	61.53 KB (0%)	3.1.3
✓hadoop103:9866 (192.168.10.103:9866)	http://hadoop103:9864	2s	2m	89.95 GB	3	61.53 KB (0%)	3.1.3

Showing 1 to 2 of 2 entries

Previous 1 Next

5) 二次修改白名单, 增加 hadoop104

```
[atguigu@hadoop102 ~]$ vim whitelist
```

修改为如下内容

```
hadoop102
hadoop103
hadoop104
```

6) 刷新 NameNode

```
[atguigu@hadoop102 ~]$ hdfs dfsadmin -refreshNodes
Refresh nodes successful
```

7) 在 web 浏览器上查看 DN, http://hadoop102:9870/dfshealth.html#tab-datanode

Show 25 entries

Search:

Node	Http Address	Last contact	Last Block Report	Capacity	Blocks	Block pool used	Version
✓hadoop102:9866 (192.168.10.102:9866)	http://hadoop102:9864	1s	0m	89.95 GB	2	40 KB (0%)	3.1.3
✓hadoop103:9866 (192.168.10.103:9866)	http://hadoop103:9864	0s	0m	89.95 GB	2	40 KB (0%)	3.1.3
✓hadoop104:9866 (192.168.10.104:9866)	http://hadoop104:9864	0s	0m	89.95 GB	2	40 KB (0%)	3.1.3

Showing 1 to 3 of 3 entries

Previous 1 Next

2.2 服役新服务器

1) 需求

随着公司业务的增长, 数据量越来越大, 原有的数据节点的容量已经不能满足存储数据的需求, 需要在原有集群基础上动态添加新的数据节点。

2) 环境准备

在 hadoop100 主机上再克隆一台 hadoop105 主机

修改 IP 地址和主机名称

```
[root@hadoop105 ~]# vim /etc/sysconfig/network-scripts/ifcfg-ens33
[root@hadoop105 ~]# vim /etc/hostname
```

拷贝 hadoop102 的/opt/module 目录和/etc/profile.d/my_env.sh 到 hadoop105

```
[atguigu@hadoop102 ~]$ scp -r module/* atguigu@hadoop105:/opt/module/
[atguigu@hadoop102 ~]$ sudo scp /etc/profile.d/my_env.sh root@hadoop105:/etc/profile.d/my_env.sh
```

```
[atguigu@hadoop105 hadoop-3.3.4]$ source /etc/profile
```

删除 hadoop105 上 Hadoop 的历史数据, data 和 log 数据

```
[atguigu@hadoop105 hadoop-3.3.4]$ rm -rf data/ logs/
```

配置 hadoop102 和 hadoop103 到 hadoop105 的 ssh 无密登录

```
[atguigu@hadoop102 .ssh]$ ssh-copy-id hadoop105
```

```
[atguigu@hadoop103 .ssh]$ ssh-copy-id hadoop105
```

3) 服役新节点具体步骤

直接启动 DataNode, 即可关联到集群

```
[atguigu@hadoop105 hadoop-3.3.4]$ hdfs --daemon start datanode
[atguigu@hadoop105 hadoop-3.3.4]$ yarn --daemon start nodemanager
```

Show

25

entries

Search:

Node	Http Address	Last contact	Last Block Report	Capacity	Blocks	Block pool used	Version
✓hadoop102:9866 (192.168.10.102:9866)	http://hadoop102:9864	2s	1m	89.95 GB	3	68 KB (0%)	3.1.3
✓hadoop103:9866 (192.168.10.103:9866)	http://hadoop103:9864	0s	33m	89.95 GB	3	68 KB (0%)	3.1.3
✓hadoop104:9866 (192.168.10.104:9866)	http://hadoop104:9864	0s	23m	89.95 GB	3	68 KB (0%)	3.1.3
✓hadoop105:9866 (192.168.10.105:9866)	http://hadoop105:9864	2s	0m	89.95 GB	0	8 KB (0%)	3.1.3

Showing 1 to 4 of 4 entries

Previous

1

Next

4) 在白名单中增加新服役的服务器

在白名单 whitelist 中增加 hadoop104、hadoop105, 并重启集群

```
[atguigu@hadoop102 hadoop]$ vim whitelist
```

修改为如下内容

```
hadoop102
hadoop103
hadoop104
hadoop105
```

分发

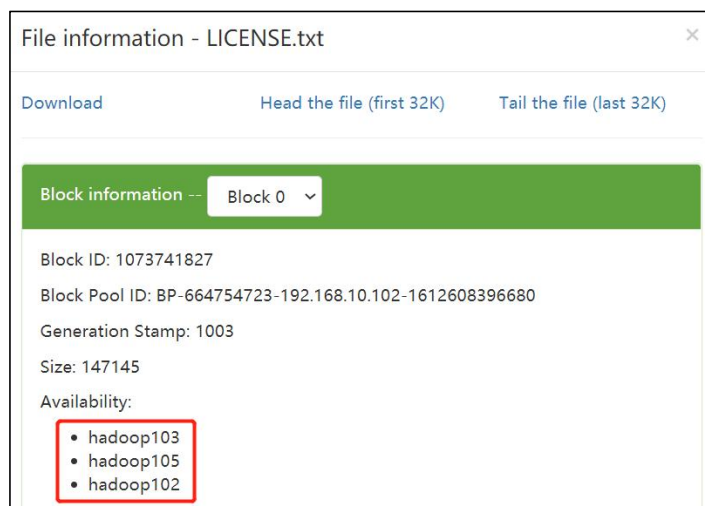
```
[atguigu@hadoop102 hadoop]$ xsync whitelist
```

刷新 NameNode

```
[atguigu@hadoop102 hadoop-3.3.4]$ hdfs dfsadmin -refreshNodes
Refresh nodes successful
```

5) 在 hadoop105 上上传文件

```
[atguigu@hadoop105 hadoop-3.3.4]$ hadoop fs -put
/opt/module/hadoop-3.3.4/LICENSE.txt /
```



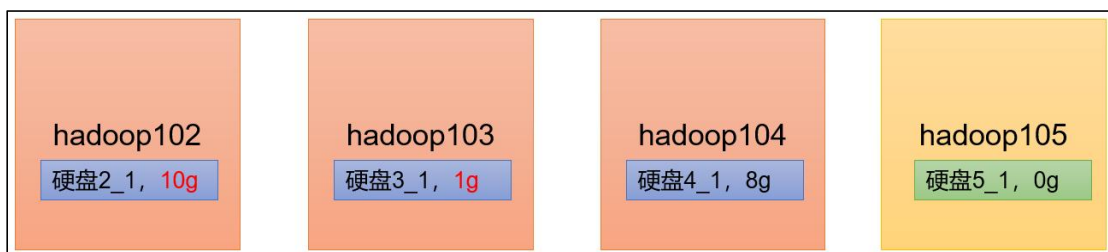
思考：如果数据不均衡（hadoop105 数据少，其他节点数据多），怎么处理？

2.3 服务器间数据均衡

1) 企业经验：

在企业开发中，如果经常在 hadoop102 和 hadoop104 上提交任务，且副本数为 2，由于数据本地性原则，就会导致 hadoop102 和 hadoop104 数据过多，hadoop103 存储的数据量小。

另一种情况，就是新服役的服务器数据量比较少，需要执行集群均衡命令。



2) 开启数据均衡命令

```
[atguigu@hadoop105 hadoop-3.3.4]$ sbin/start-balancer.sh -threshold 10
```

对于参数 10，代表的是集群中各个节点的磁盘空间利用率相差不超过 10%，可根据实际情况进行调整。

3) 停止数据均衡命令

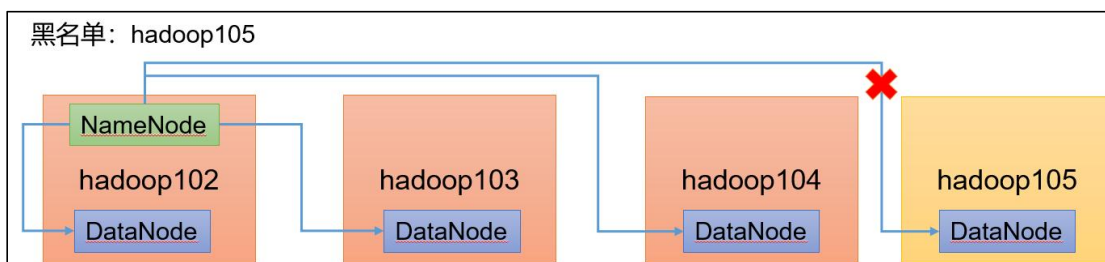
```
[atguigu@hadoop105 hadoop-3.3.4]$ sbin/stop-balancer.sh
```

注意：由于 HDFS 需要启动单独的 Rebalance Server 来执行 Rebalance 操作，所以尽量不要在 NameNode 上执行 start-balancer.sh，而是找一台比较空闲的机器。

2.4 黑名单退役服务器

黑名单：表示在黑名单的主机 IP 地址不可以，用来存储数据。

企业中：配置黑名单，用来退役服务器。



黑名单配置步骤如下：

1) 编辑/opt/module/hadoop-3.3.4/etc/hadoop 目录下的 blacklist 文件

```
[atguigu@hadoop102 ~]$ vim blacklist
```

添加如下主机名称（要退役的节点）

```
hadoop105
```

第一次添加黑名单必须重启集群，不是第一次，只需要刷新 NameNode 节点即可

```
[atguigu@hadoop102 ~]$ hdfs dfsadmin -refreshNodes
Refresh nodes successful
```

2) 检查 Web 浏览器，退役节点的状态为 decommission in progress（退役中），说明数据节点正在复制块到其他节点



hadoop105:9866 (192.168.10.105:9866)	http://hadoop105:9864	1s	9m	89.95 GB	0	16 KB (0%)	3.1.3
---	-----------------------	----	----	----------	---	------------	-------

3) 等待退役节点状态为 decommissioned（所有块已经复制完成），停止该节点及节点资源管理器。注意：如果副本数是 3，服役的节点小于等于 3，是不能退役成功的，需要修改副本数后才能退役

hadoop105:9866 (192.168.10.105:9866)	http://hadoop105:9864	0s	4m	89.95 GB	2	152.87 KB (0%)	3.1.3
---	-----------------------	----	----	----------	---	----------------	-------

```
[atguigu@hadoop105 ~]$ hdfs --daemon stop datanode
stopping datanode
[atguigu@hadoop105 ~]$ yarn --daemon stop nodemanager
stopping nodemanager
```

4) 如果数据不均衡，可以用命令实现集群的再平衡

```
[atguigu@hadoop102 ~]$ sbin/start-balancer.sh -threshold 10
```

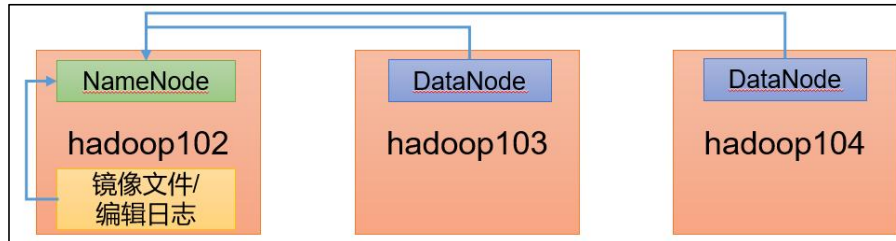
第 3 章 HDFS—安全模式（故障排除）

注意：采用三台服务器即可，恢复到 Yarn 开始的服务器快照。

1) 安全模式：文件系统只接受读数据请求，而不接受删除、修改等变更请求

2) 进入安全模式场景

- NameNode 在加载镜像文件和编辑日志期间处于安全模式
- NameNode 再接收 DataNode 注册时，处于安全模式



3) 退出安全模式条件

- `dfs.namenode.safemode.min.datanodes`:最小可用 datanode 数量，默认 0
- `dfs.namenode.safemode.threshold-pct`:副本数达到最小要求的 block 占系统总 block 数的百分比，默认 0.999f。（只允许丢一个块）
- `dfs.namenode.safemode.extension`:稳定时间，默认值 30000 毫秒，即 30 秒

4) 基本语法

集群处于安全模式，不能执行重要操作（写操作）。集群启动完成后，自动退出安全模式。

- `bin/hdfs dfsadmin -safemode get` （功能描述：查看安全模式状态）
- `bin/hdfs dfsadmin -safemode enter` （功能描述：进入安全模式状态）
- `bin/hdfs dfsadmin -safemode leave` （功能描述：离开安全模式状态）
- `bin/hdfs dfsadmin -safemode wait` （功能描述：等待安全模式状态）

5) 案例 1：启动集群进入安全模式

重新启动集群

```
[atguigu@hadoop102 subdir0]$ myhadoop.sh stop
[atguigu@hadoop102 subdir0]$ myhadoop.sh start
```

集群启动后，立即来到集群上删除数据，提示集群处于安全模式

Hadoop
Overview
Datanodes
Datanode Volume Failures
Snapshot
Startup Progress
Utilities

Browse Directory

Cannot delete /NOTICE.txt. Name node is in safe mode. The reported blocks 0 needs additional 4 blocks to reach the threshold 0.9990 of total blocks 5. The minimum number of live datanodes is not required. Safe mode will be turned off automatically once the thresholds have been reached.
NamenodeHostName:hadoop102

/
Go!

Show 25 entries
Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	atguigu	supergroup	21.35 KB	Feb 14 10:58	3	128 MB	NOTICE.txt

6) 案例 2: 磁盘修复

需求: 数据块损坏, 进入安全模式, 如何处理

分 别 进 入 hadoop102 、 hadoop103 、 hadoop104 的
/opt/module/hadoop-3.3.4/data/dfs/data/current/BP-1015489500-192.168.10.102-1611909480872/
current/finalized/subdir0/subdir0 目录, 统一删除某 2 个块信息

```
[atguigu@hadoop102 subdir0]$ pwd
/opt/module/hadoop-3.3.4/data/dfs/data/current/BP-1015489500-192.168.10.102-1611909480872/current/finalized/subdir0/subdir0

[atguigu@hadoop102 subdir0]$ rm -rf blk_1073741847
blk_1073741847_1023.meta
[atguigu@hadoop102 subdir0]$ rm -rf blk_1073741865
blk_1073741865_1042.meta
```

说明: hadoop103/hadoop104 重复执行以上命令

重新启动集群

```
[atguigu@hadoop102 subdir0]$ myhadoop.sh stop
[atguigu@hadoop102 subdir0]$ myhadoop.sh start
```

观察 <http://hadoop102:9870/dfshealth.html#tab-overview>

Summary

Security is off.

Safe mode is ON The reported blocks 5 needs additional 1 blocks to reach the threshold 0.9990 of total blocks 7. The minimum number of live datanodes is not required. Safe mode will be turned off automatically once the thresholds have been reached.

说明: 安全模式已经打开, 块的数量没有达到要求。

离开安全模式

```
[atguigu@hadoop102 subdir0]$ hdfs dfsadmin -safemode get
Safe mode is ON
[atguigu@hadoop102 subdir0]$ hdfs dfsadmin -safemode leave
Safe mode is OFF
```

观察 <http://hadoop102:9870/dfshealth.html#tab-overview>



将元数据删除

```
[atguigu@hadoop102 hadoop-3.3.4]$ hdfs fsck -delete /
```

观察 <http://hadoop102:9870/dfshealth.html#tab-overview>, 集群已经正常

7) 案例 3:

需求: 模拟等待安全模式。

查看当前模式

```
[atguigu@hadoop102 hadoop-3.3.4]$ hdfs dfsadmin -safemode get
Safe mode is OFF
```

先进入安全模式

```
[atguigu@hadoop102 hadoop-3.3.4]$ bin/hdfs dfsadmin -safemode enter
```

创建并执行下面的脚本

在/opt/module/hadoop-3.3.4 路径上, 编辑一个脚本 safemode.sh。

```
[atguigu@hadoop102 hadoop-3.3.4]$ vim safemode.sh

#!/bin/bash
hdfs dfsadmin -safemode wait
hdfs dfs -put /opt/module/hadoop-3.3.4/README.txt /

[atguigu@hadoop102 hadoop-3.3.4]$ chmod 777 safemode.sh

[atguigu@hadoop102 hadoop-3.3.4]$ ./safemode.sh
```

再打开一个窗口, 执行

```
[atguigu@hadoop102 hadoop-3.3.4]$ bin/hdfs dfsadmin -safemode leave
```

再观察上一个窗口

```
Safe mode is OFF
```

HDFS 集群上已经有上传的数据了



第 4 章 Hadoop 数据压缩

4.1 概述

1) 压缩的好处和坏处

压缩的优点：以减少磁盘 IO、减少磁盘存储空间。

压缩的缺点：增加 CPU 开销。

2) 压缩原则

运算密集型的 Job，少用压缩。

IO 密集型的 Job，多用压缩。

4.2 MR 支持的压缩编码

1) 压缩算法对比介绍

压缩格式	Hadoop 自带?	算法	文件扩展名	是否可切片	换成压缩格式后，原来的程序是否需要修改
DEFLATE	是，直接使用	DEFLATE	.deflate	否	和文本处理一样，不需要修改
Gzip	是，直接使用	DEFLATE	.gz	否	和文本处理一样，不需要修改
bzip2	是，直接使用	bzip2	.bz2	是	和文本处理一样，不需要修改
LZO	否，需要安装	LZO	.lzo	是	需要建索引，还需要指定输入格式
Snappy	是，直接使用	Snappy	.snappy	否	和文本处理一样，不需要修改

2) 压缩性能的比较

压缩算法	原始文件大小	压缩文件大小	压缩速度	解压速度
gzip	8.3GB	1.8GB	17.5MB/s	58MB/s
bzip2	8.3GB	1.1GB	2.4MB/s	9.5MB/s
LZO	8.3GB	2.9GB	49.3MB/s	74.6MB/s

<http://google.github.io/snappy/>

Snappy is a compression/decompression library. It **does not aim for maximum compression**, or compatibility with any other compression library; instead, it **aims for very high speeds** and reasonable compression. For instance, compared to the fastest mode of zlib, Snappy is an order of

magnitude faster for most inputs, but the resulting compressed files are anywhere from 20% to 100% bigger. On a single core of a Core i7 processor in 64-bit mode, Snappy **compresses** at about **250 MB/sec** or more and **decompresses** at about **500 MB/sec** or more.

4.3 压缩方式选择

压缩方式选择时重点考虑：**压缩/解压缩速度**、**压缩率（压缩后存储大小）**、**压缩后是否可以支持切片**。

4.3.1 Gzip 压缩

优点：压缩率比较高。

缺点：不支持 Split；压缩/解压速度一般。

4.3.2 Bzip2 压缩

优点：压缩率高；支持 Split。

缺点：压缩/解压速度慢。

4.3.3 Lzo 压缩

优点：压缩/解压速度比较快；支持 Split。

缺点：压缩率一般；想支持切片需要额外创建索引。

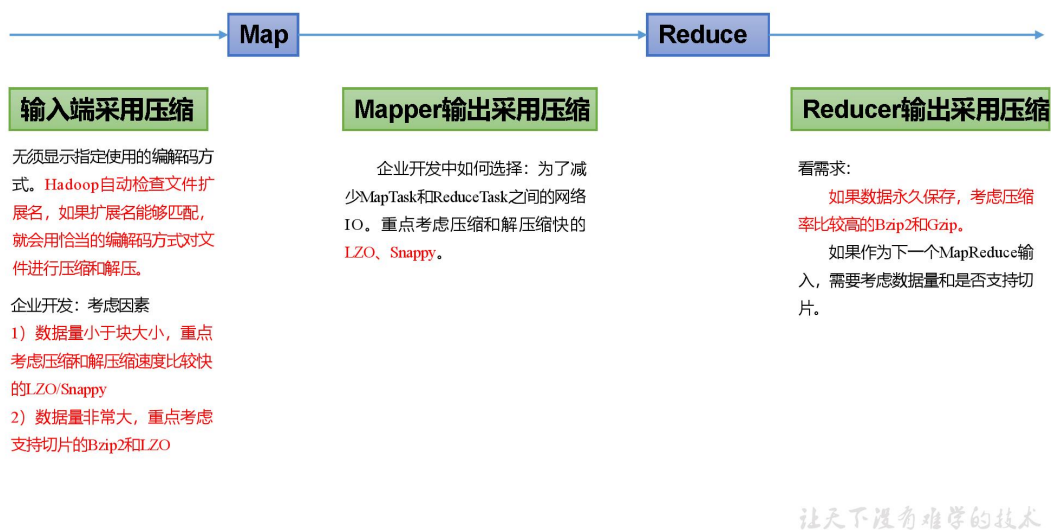
4.3.4 Snappy 压缩

优点：压缩和解压缩速度快。

缺点：不支持 Split；压缩率一般。

4.3.5 压缩位置选择

压缩可以在 MapReduce 作用的任意阶段启用。



4.4 压缩参数配置

1) 为了支持多种压缩/解压缩算法, Hadoop 引入了编码/解码器

压缩格式	对应的编码/解码器
DEFLATE	org.apache.hadoop.io.compress.DefaultCodec
gzip	org.apache.hadoop.io.compress.GzipCodec
bzip2	org.apache.hadoop.io.compress.BZip2Codec
LZO	com.hadoop.compression.lzo.LzopCodec
Snappy	org.apache.hadoop.io.compress.SnappyCodec

2) 要在 Hadoop 中启用压缩, 可以配置如下参数

参数	默认值	阶段	建议
io.compression.codecs (在 core-site.xml 中配置)	无, 这个需要在命令行输入 <code>hadoop checknative</code> 查看	输入压缩	Hadoop 使用文件扩展名判断是否支持某种编解码器
mapreduce.map.output.compress (在 mapred-site.xml 中配置)	false	mapper 输出	这个参数设为 true 启用压缩

<code>mapreduce.map.output.compress.codec</code> (在 <code>mapred-site.xml</code> 中配置)	<code>org.apache.hadoop.io.compress.DefaultCodec</code>	mapper 输出	企业多使用 LZO 或 Snappy 编解码器在此阶段压缩数据
<code>mapreduce.output.fileoutputformat.compress</code> (在 <code>mapred-site.xml</code> 中配置)	false	reducer 输出	这个参数设为 true 启用压缩
<code>mapreduce.output.fileoutputformat.compress.codec</code> (在 <code>mapred-site.xml</code> 中配置)	<code>org.apache.hadoop.io.compress.DefaultCodec</code>	reducer 输出	使用标准工具或者编解码器, 如 gzip 和 bzip2

4.5 压缩实操案例

4.5.1 Map 输出端采用压缩

即使你的 MapReduce 的输入输出文件都是未压缩的文件, 你仍然可以对 Map 任务的中间结果输出做压缩, 因为它要写在硬盘并且通过网络传输到 Reduce 节点, 对其压缩可以提高很多性能, 这些工作只要设置两个属性即可, 我们来看下代码怎么设置。

1) 给大家提供的 Hadoop 源码支持的压缩格式有: **BZip2Codec**、**DefaultCodec**

```
package com.atguigu.mapreduce.compress;
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.compress.BZip2Codec;
import org.apache.hadoop.io.compress.CompressionCodec;
import org.apache.hadoop.io.compress.GzipCodec;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {

    public static void main(String[] args) throws IOException,
        ClassNotFoundException, InterruptedException {

        Configuration conf = new Configuration();

        // 开启 map 端输出压缩
        conf.setBoolean("mapreduce.map.output.compress", true);

        // 设置 map 端输出压缩方式
```

```
        conf.setClass("mapreduce.map.output.compress.codec",
BZip2Codec.class, CompressionCodec.class);

        Job job = Job.getInstance(conf);

        job.setJarByClass(WordCountDriver.class);

        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        boolean result = job.waitForCompletion(true);

        System.exit(result ? 0 : 1);
    }
}
```

2) Mapper 保持不变

```
package com.atguigu.mapreduce.compress;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper extends Mapper<LongWritable, Text, Text,
IntWritable>{

    Text k = new Text();
    IntWritable v = new IntWritable(1);

    @Override
    protected void map(LongWritable key, Text value, Context
context)throws IOException, InterruptedException {

        // 1 获取一行
        String line = value.toString();

        // 2 切割
        String[] words = line.split(" ");

        // 3 循环写出
        for(String word:words){
            k.set(word);
            context.write(k, v);
        }
    }
}
```

3) Reducer 保持不变

```
package com.atguigu.mapreduce.compress;
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer extends Reducer<Text, IntWritable, Text,
IntWritable>{

    IntWritable v = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException
    {

        int sum = 0;

        // 1 汇总
        for(IntWritable value:values){
            sum += value.get();
        }

        v.set(sum);

        // 2 输出
        context.write(key, v);
    }
}
```

4.5.2 Reduce 输出端采用压缩

基于 WordCount 案例处理。

1) 修改驱动

```
package com.atguigu.mapreduce.compress;
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.compress.BZip2Codec;
import org.apache.hadoop.io.compress.DefaultCodec;
import org.apache.hadoop.io.compress.GzipCodec;
import org.apache.hadoop.io.compress.Lz4Codec;
import org.apache.hadoop.io.compress.SnappyCodec;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {

    public static void main(String[] args) throws IOException,
        ClassNotFoundException, InterruptedException {
```

```
Configuration conf = new Configuration();

Job job = Job.getInstance(conf);

job.setJarByClass(WordCountDriver.class);

job.setMapperClass(WordCountMapper.class);
job.setReducerClass(WordCountReducer.class);

job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

FileInputFormat.setInputPaths(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

// 设置 reduce 端输出压缩开启
FileOutputFormat.setCompressOutput(job, true);

// 设置压缩的方式
FileOutputFormat.setOutputCompressorClass(job,
BZip2Codec.class);
// FileOutputFormat.setOutputCompressorClass(job,
GzipCodec.class);
// FileOutputFormat.setOutputCompressorClass(job,
DefaultCodec.class);

boolean result = job.waitForCompletion(true);

System.exit(result?0:1);
}
```

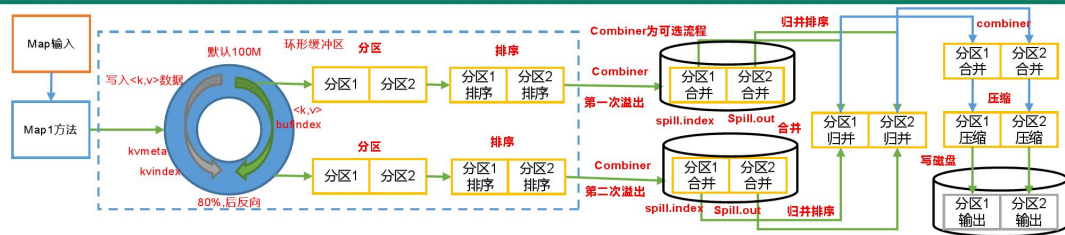
2) Mapper 和 Reducer 保持不变 (详见 4.5.1)

第 5 章 Hadoop 企业优化

5.1 MapReduce 优化方法

MapReduce 优化方法主要从六个方面考虑：数据输入、Map 阶段、Reduce 阶段、IO 传输、数据倾斜问题和常用的调优参数。

MapReduce优化 (上)



1) 合并小文件

采用 `CombineTextInputFormat` 切片规则

2) 自定义分区，减少数据倾斜；

定义类，实现 `Partitioner` 接口，重写 `getPartition` 方法

3) 减少溢写的次数

`mapreduce.task.io.sort.mb`

Shuffle 的环形缓冲区大小，默认 100m，可以提高到 200m

`mapreduce.map.sort.spill.percent`

环形缓冲区溢出的阈值，默认 80%，可以提高到 90%

4) 增加每次 Merge 合并次数

`mapreduce.task.io.sort.factor` 默认 10，可以提高到 20

5) 在不影响业务结果的前提下可以提前采用 Combiner

`job.setCombinerClass(xxxReducer.class);`

6) 为了减少磁盘 IO，可以采用 Snappy 或者 LZO 压缩
`conf.setBoolean("mapreduce.map.output.compress", true);`
`conf.setClass("mapreduce.map.output.compress.codec",
 SnappyCodec.class, CompressionCodec.class);`

7) `mapreduce.map.memory.mb` 默认 Map Task 内存上限 1024MB。

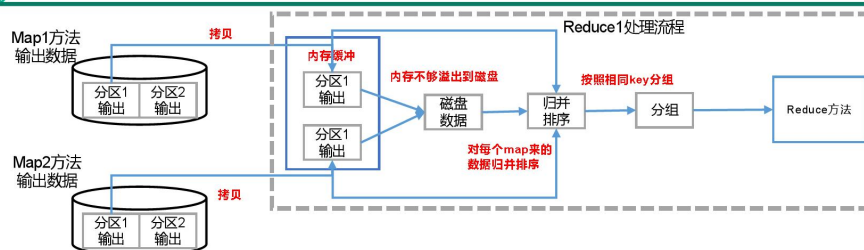
可以根据 128m 数据对应 1G 内存原则提高该内存。

8) `mapreduce.map.cpu.vcores` 默认 Map Task 的 CPU 核数 1。计算密集型任务可以增加 CPU 核数

9) 异常重试

`mapreduce.map.maxattempts` 每个 Map Task 最大重试次数，一旦重试次数超过该值，则认为 Map Task 运行失败，默认值：4。根据机器性能适当提高。

MapReduce优化 (下)



1) `mapreduce.reduce.shuffle.parallelcopies` 每个 Reduce 去 Map 中拉取数据的并行数，默认值是 5。可以提高到 10。

2) `mapreduce.reduce.shuffle.input.buffer.percent`

Buffer 大小占 Reduce 可用内存的比例，默认值 0.7。可以提高到 0.8

3) `mapreduce.reduce.shuffle.merge.percent` Buffer 中的数据达到多少比例开始写入磁盘，默认值 0.66。可以提高到 0.75

4) `mapreduce.reduce.memory.mb` 默认 Reduce Task 内存上限 1024MB，根据 128m 数据对应 1G 内存原则，适当提高内存到 4-6G

5) `mapreduce.reduce.cpu.vcores` 默认 Reduce Task 的 CPU 核数 1 个。可以提高到 2-4 个

6) `mapreduce.reduce.maxattempts` 每个 Reduce Task 最大重试次数，一旦重试次数超过该值，则认为 Map Task 运行失败，默认值：4。

7) `mapreduce.job.reduce.slowstart.completedmaps` 当 Map Task 完成的比例达到该值后才会为 Reduce Task 申请资源。默认是 0.05。

8) `mapreduce.task.timeout` 如果一个 Task 在一定时间内没有任何进入，即不会读取新的数据，也没有输出数据，则认为该 Task 处于 Block 状态，可能是卡住了，也许永远会卡住，为了防止因为用户程序永远 Block 住不退出，则强制设置了一个该超时时间（单位毫秒），默认是 600000（10 分钟）。如果你的程序对每条输入数据的处理时间过长，建议将该参数调大。

9) 如果可以不用 Reduce，尽可能不用

5.2 数据倾斜问题



数据倾斜问题

1. 数据倾斜现象

数据频率倾斜——某一个区域的数据量要远远大于其他区域。

数据大小倾斜——部分记录的大小远远大于平均值。

让天下没有难学的技术



2. 减少数据倾斜的方法

方法1：抽样和范围分区

可以通过对原始数据进行抽样得到的结果集来预设分区边界值。

方法2：自定义分区

基于输出键的背景知识进行自定义分区。例如，如果Map输出键的单词来源于一本书。且其中某几个专业词汇较多。那么就可以自定义分区将这这些专业词汇发送给固定的一部分Reduce实例。而将其他的都发送给剩余的Reduce实例。

方法3：Combiner

使用Combiner可以大量地减小数据倾斜。在可能的情况下，Combine的目的就是聚合并精简数据。

方法4：采用Map Join，尽量避免Reduce Join。

让天下没有难学的技术

5.3 Hadoop 小文件优化方法

5.3.1 Hadoop 小文件弊端

HDFS 上每个文件都要在 NameNode 上创建对应的元数据，这个元数据的大小约为 150byte，这样当小文件比较多时，就会产生很多的元数据文件，一方面会大量占用 NameNode 的内存空间，另一方面就是元数据文件过多，使得寻址索引速度变慢。

小文件过多，在进行 MR 计算时，会生成过多切片，需要启动过多的 MapTask。每个 MapTask 处理的数据量小，导致 MapTask 的处理时间比启动时间还小，白白消耗资源。

5.3.2 Hadoop 小文件解决方案

1) Hadoop Archive (HAR 归档)

是一个高效的将小文件放入 HDFS 块中的文件存档工具，能够将多个小文件打包成一个 HAR 文件，从而达到减少 NameNode 的内存使用。

2) CombineTextInputFormat

CombineTextInputFormat 用于将多个小文件在切片过程中生成一个单独的切片或者少量的切片。

第 6 章 Hadoop 扩展

6.1 集群间数据拷贝

1) scp 实现两个远程主机之间的文件复制

- `scp -r hello.txt root@hadoop103:/user/atguigu/hello.txt` // 推 push
- `scp -r root@hadoop103:/user/atguigu/hello.txt hello.txt` // 拉 pull
- `scp -r root@hadoop103:/user/atguigu/hello.txt root@hadoop104:/user/atguigu` // 是通过本地主机中转实现两个远程主机的文件复制；如果在两个远程主机之间 ssh 没有配置的情况下可以使用该方式。

2) 采用 distcp 命令实现两个 Hadoop 集群之间的递归数据复制

```
[atguigu@hadoop102 hadoop-3.3.4]$ bin/hadoop distcp
hdfs://hadoop102:8020/user/atguigu/hello.txt
hdfs://hadoop105:8020/user/atguigu/hello.txt
```

6.2 小文件存档

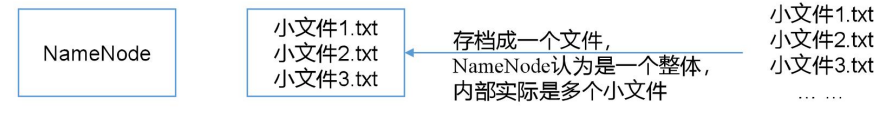

小文件存档


1、HDFS存储小文件弊端

每个文件均按块存储，每个块的元数据存储在NameNode的内存中，因此HDFS存储小文件会非常低效。因为大量的的小文件会耗尽NameNode中的大部分内存。但注意，存储小文件所需要的磁盘容量和数据块的大小无关。例如，一个1MB的文件设置为128MB的块存储，实际使用的是1MB的磁盘空间，而不是128MB。

2、解决存储小文件办法之一

HDFS存档文件或HAR文件，是一个更高效的文件存档工具，它将文件存入HDFS块，在减少NameNode内存使用的同时，允许对文件进行透明的访问。具体说来，HDFS存档文件对内还是一个一个独立文件，对NameNode而言却是一个整体，减少了NameNode的内存。



让天下没有难学的技术

案例实操

需要启动 YARN 进程

```
[atguigu@hadoop102 hadoop-3.3.4]$ start-yarn.sh
```

归档文件

把/user/atguigu/input 目录里面的所有文件归档成一个叫 input.har 的归档文件，并把归档后文件存储到/user/atguigu/output 路径下。

```
[atguigu@hadoop102 hadoop-3.3.4]$ hadoop archive -archiveName input.har -p /user/atguigu/input /user/atguigu/output
```

查看归档

```
[atguigu@hadoop102 hadoop-3.3.4]$ hadoop fs -ls /user/atguigu/output/input.har
[atguigu@hadoop102 hadoop-3.3.4]$ hadoop fs -ls har:///user/atguigu/output/input.har
```

解归档文件

```
[atguigu@hadoop102 hadoop-3.3.4]$ hadoop fs -cp har:///user/atguigu/output/input.har/* /user/atguigu
```

6.3 回收站

开启回收站功能，可以将删除的文件在不超时的情况下，恢复原数据，起到防止误删除、备份等作用。

1) 回收站参数设置及工作机制


回收站


一、开启回收站功能参数说明：

- 默认值fs.trash.interval=0，0表示禁用回收站;其他值表示设置文件的存活时间。
- 默认值fs.trash.checkpoint.interval=0，检查回收站的间隔时间。如果该值为0，则该值设置和fs.trash.interval的参数值相等。
- 要求fs.trash.checkpoint.interval<=fs.trash.interval。

二、回收站工作机制：

被删除的文件

回收站

被删除的文件

设置文件存活时间
fs.trash.interval=60

检查回收站的间隔时间

fs.trash.checkpoint.interval=10



让天下没有难学的技术

2) 启用回收站

修改 core-site.xml，配置垃圾回收时间为 1 分钟。

```
<property>
  <name>fs.trash.interval</name>
```

```
<value>1</value>
</property>
```

3) 查看回收站

回收站目录在 `hdfs` 集群中的路径: `/user/atguigu/.Trash/...`

4) 通过程序删除的文件不会经过回收站, 需要调用 `moveToTrash()`才进入回收站

```
@Test
public void testRm() throws IOException, InterruptedException {
    URI uri = URI.create("hdfs://hadoop102:8020");
    conf = new Configuration();
    conf.set("fs.trash.interval", "1");
    conf.set("fs.trash.checkpoint.interval", "1");
    String user = "atguigu";
    fileSystem = FileSystem.get(uri, conf, user);
    //创建一个回收站对象
    Trash trash = new Trash(fileSystem, conf);
    trash.moveToTrash(new Path("/out10"));
}
```

5) 通过网页上直接删除的文件也不会走回收站。

6) 只有在命令行利用 `hadoop fs -rm` 命令删除的文件才会走回收站。

```
[atguigu@hadoop102 hadoop-3.3.4]$ hadoop fs -rm -r
/user/atguigu/input
2020-07-14 16:13:42,643 INFO fs.TrashPolicyDefault: Moved:
'hdfs://hadoop102:8020/user/atguigu/input' to trash at:
hdfs://hadoop102:8020/user/atguigu/.Trash/Current/user/atguigu/in
put
```

7) 恢复回收站数据

```
[atguigu@hadoop102 hadoop-3.3.4]$ hadoop fs -mv
/user/atguigu/.Trash/Current/user/atguigu/input
/user/atguigu/input
```